INTERNET PROGRAMMING
LAB EXPERIMENTS

Experiments 7 to 12

Name	Dhuneshwaran V
Register Number	192324169
Course Name:	Internet Programming
Course Code	CSA4301
Lab Details:	EXP 7 to 12

EXPERIMENT 7

TITLE: Online Beauty Parlour Appointment Booking System

AIM: To develop a simple web application that allows customers to view beauty services, select a stylist, book an appointment, manage reservations, and submit feedback.

PROCEDURE:

1. Create an HTML page with salon services, an appointment form, a manager dashboard, and a feedback section.
2. Add form fields for customer name, phone number, service, stylist, date, time, and payment option.
3. Use CSS to style the header, service cards, form, buttons, and appointment details.
4. Use JavaScript to validate the booking, calculate the full or 50% deposit amount, and prevent duplicate stylist time slots.
5. Display booked appointments with options to edit the date or cancel the reservation. Allow customers to submit feedback.

SHORT CODE:

```
// Check whether the selected stylist is already booked
const unavailable = appointments.some(a =>
    a.date === date &&
    a.time === time &&
    a.stylist === stylist &&
    a.status !== "Cancelled"
);

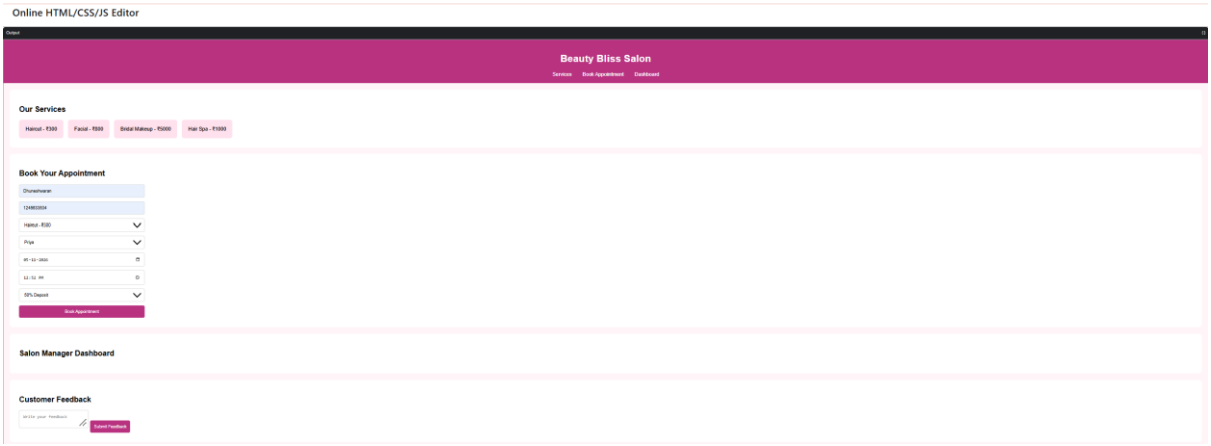
if (unavailable) {
    alert("This time slot is already booked!");
    return;
}

// Calculate full payment or 50% deposit
const amount = payment === "full" ? price : price * 0.5;

appointments.push({
    name, phone, service, stylist, date, time,
    amount, status: "Confirmed"
});

displayAppointments();
```

OUTCOME :



RESULT:

The Beauty Bliss Salon booking page was created successfully. Customers can select a service and stylist, book a date and time, view appointments, edit or cancel a reservation, and submit feedback. The payment amount is calculated for full payment or a 50% deposit.

EXPERIMENT 8

TITLE: MedCare Hospital Appointment Booking System

AIM: To develop an online hospital appointment system that enables patients to search for hospitals, view doctors, book or cancel appointments, submit check-in details, and provide reviews.

PROCEDURE:

6. Create the MedCare home page with patient registration, hospital search, doctor list, appointment booking, check-in, reviews, and admin dashboard sections.
7. Add input fields to collect patient details and search for a hospital by location.
8. Display available doctors with their specialties and provide date and time fields for booking.
9. Use JavaScript to validate appointment details and check whether the selected doctor's time slot is already booked.
10. Display appointment history and total appointment count. Add cancellation, check-in, review, and Google Maps link features.

SHORT CODE:

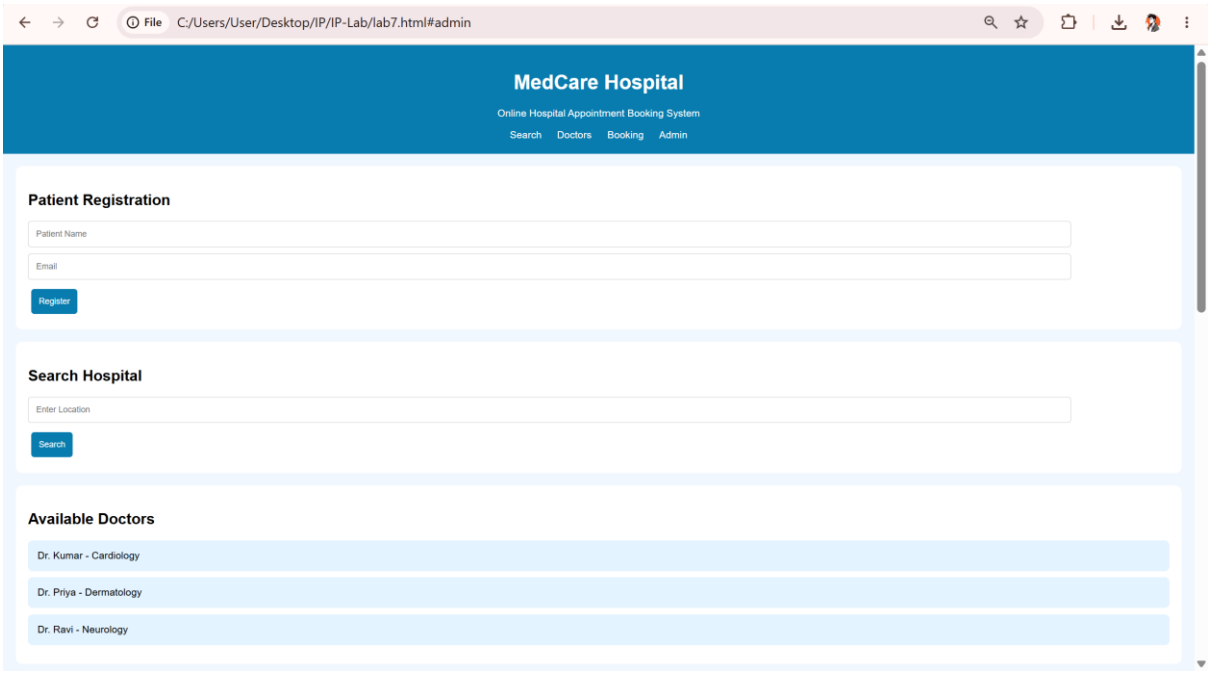
```
// Book an appointment after checking the slot
const exists = appointments.some(a =>
    a.doctor === doctor &&
    a.date === date &&
    a.time === time &&
    a.status === "Booked"
);

if (exists) {
    alert("Appointment slot already booked!");
    return;
}

appointments.push({
    patient, doctor, date, time,
    status: "Booked"
});

displayAppointments();
```

OUTCOME :



RESULT:

The MedCare appointment booking page was created successfully. Patients can search for a hospital, view doctors, book and cancel appointments, submit check-in details, and write reviews. The admin section displays appointment records and the total count.

EXPERIMENT 9

TITLE: TechSavvy Online Shopping Platform

AIM: To develop an online shopping website where users can browse gadget categories, search products, add or remove items from a cart, checkout, and view order history, while the admin can add and delete products.

PROCEDURE:

11. Create the TechSavvy home page with login, product categories, search, shopping cart, order history, and admin sections.
12. Define sample products with their names, prices, and categories such as mobiles, laptops, and accessories.
13. Display products based on the category selected by the user and provide a search option.
14. Use JavaScript to add products to the cart, calculate the total price, and remove items from the cart.
15. Validate login and cart contents before checkout. Store a simple order record and provide admin options to add or delete products.

SHORT CODE:

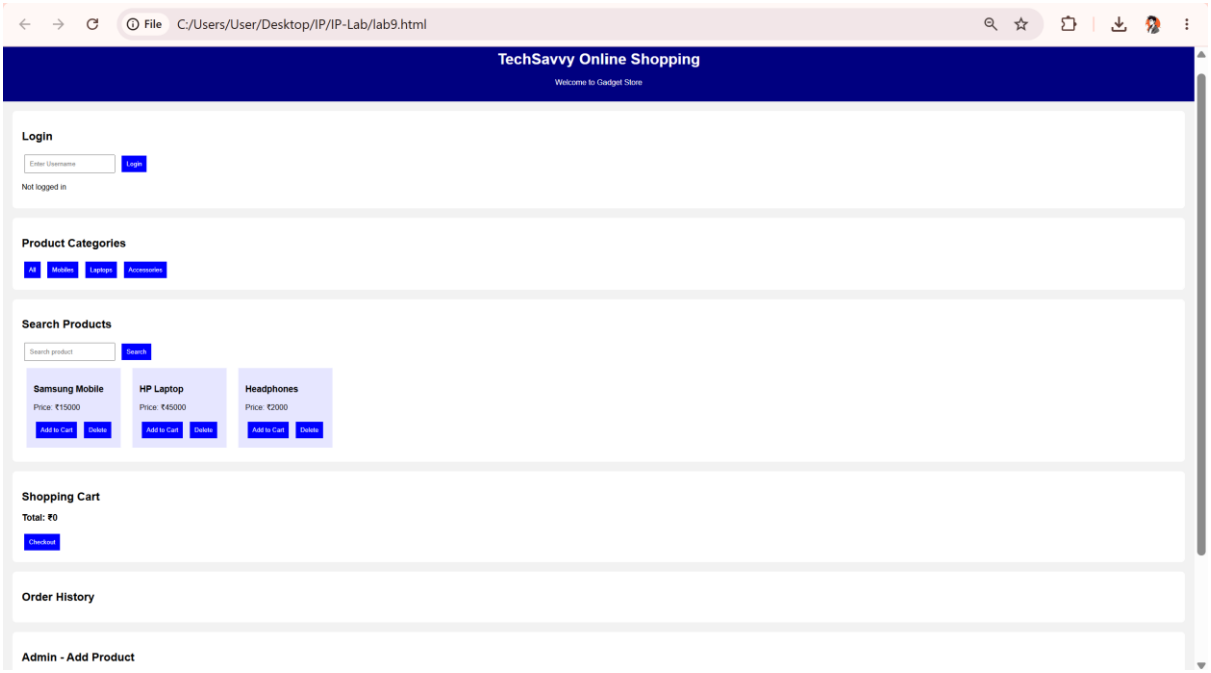
```
// Add a selected product to the cart
function addCart(index) {
    cart.push(products[index]);
    displayCart();
}

// Calculate cart total and display items
function displayCart() {
    let total = 0;
    let html = "";
```

```
    cart.forEach((p, i) => {
        total += p.price;
        html += `<p>${p.name} - ₹${p.price}
                 <button onclick="removeCart(${i})">Remove</button></p>`;
    });

    document.getElementById("cart").innerHTML = html;
    document.getElementById("total").innerText = total;
}
```

OUTCOME :



RESULT:

The TechSavvy shopping website was created successfully. Users can log in, browse product categories, search products, add or remove products from the cart, calculate the total, checkout, and view order history. The admin can add and delete products.

EXPERIMENT 10

TITLE: StyleHub Online Fashion Shopping Platform

AIM: To develop an e-commerce website for clothing, footwear, and accessories with category-based browsing, product search, cart management, checkout, order history, and admin product management.

PROCEDURE:

16. Create the StyleHub page with login, shopping categories, product listing, cart, order history, and admin panel.
17. Prepare sample products under Men, Women, Kids, and Accessories categories with price and description.
18. Use JavaScript to filter products by category and search products by name or category.
19. Implement product details, add-to-cart, remove-from-cart, and total price calculation.

20. Allow logged-in users to checkout and view order history. Provide admin functions to add products, edit prices, and delete products.

SHORT CODE:

```
// Filter products by category
function showProducts(category) {
    const result = products.filter(p =>
        category === "All" || p.category === category
    );

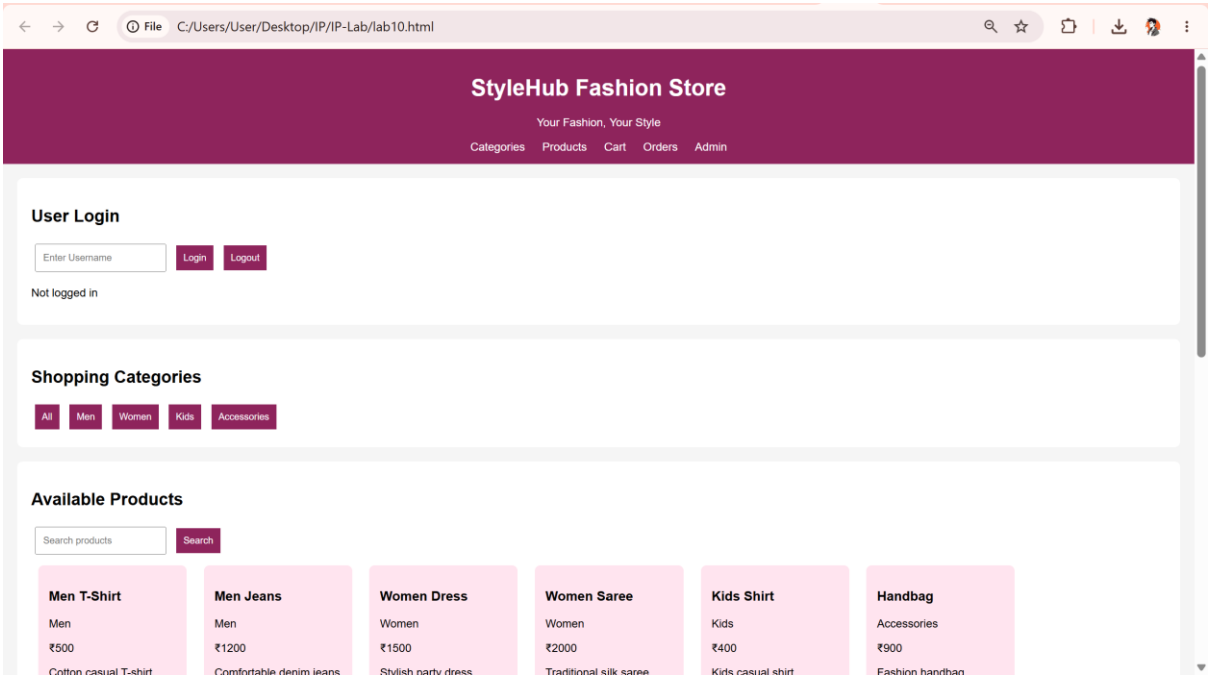
    displayProducts(result);
}

// Add product to cart
function addCart(index) {
    cart.push(products[index]);
    displayCart();
}

// Update product price (admin)
function editPrice(index) {
    const price = prompt("Enter new price");

    if (price !== null && Number(price) > 0) {
        products[index].price = Number(price);
        showProducts("All");
    }
}
```

OUTCOME :



RESULT:

The StyleHub fashion shopping website was created successfully. Users can browse Men, Women, Kids, and Accessories categories, search and view products, manage the cart, checkout, and view orders. Admin functions support adding, deleting, and updating product prices.

EXPERIMENT 11

TITLE: Home Finder – Online Real Estate Property Booking System

AIM: To develop a real estate website that helps buyers search for houses, lands, apartments, and commercial properties based on location, budget, and property type, and view property locations on Google Maps.

PROCEDURE:

21. Create the Home Finder page with buyer login, property search, property listings, favorites, agent search, and visitor records.
22. Prepare sample property records containing property name, location, type, price, and agent details.
23. Provide search fields for location, maximum budget, and property type.
24. Use JavaScript to filter properties according to the buyer's search criteria and display matching results.
25. Allow buyers to save search criteria and favorite properties. Add agent/company search and Google Maps links for property locations.
26. Record visitor names when users log in and display the visitor list.

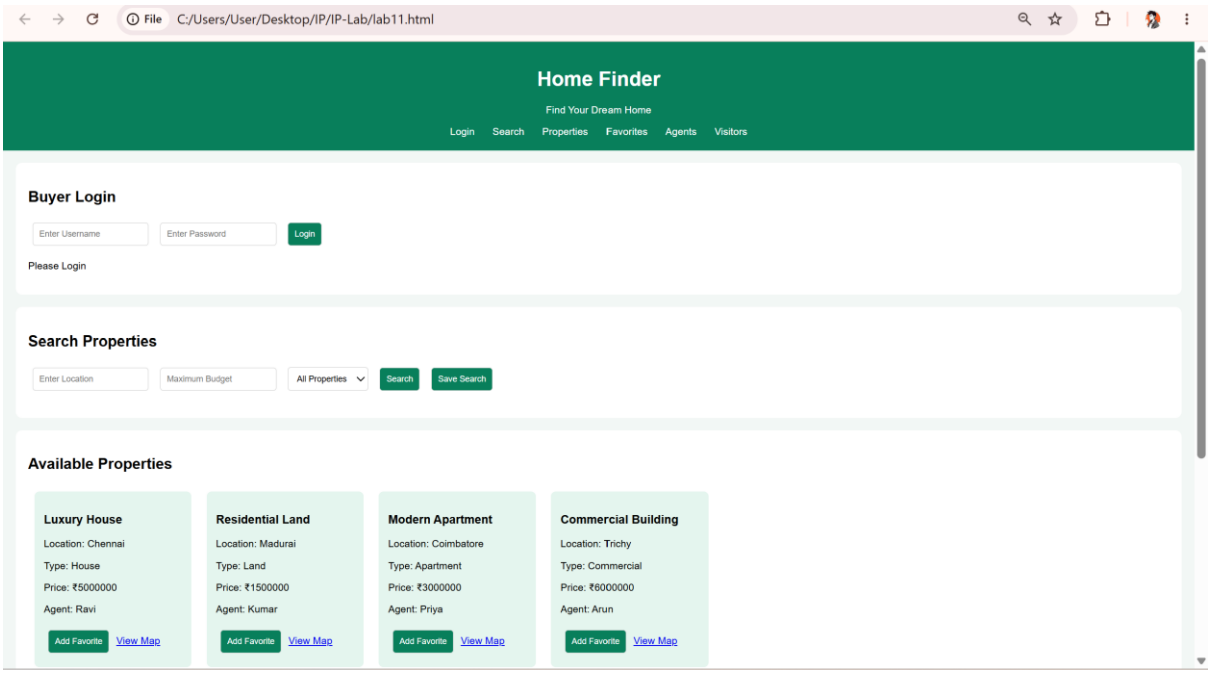
SHORT CODE:

```
// Search properties using location, budget, and type
const result = properties.filter(p =>
    (!location || p.location.toLowerCase().includes(location)) &&
    (!budget || p.price <= budget) &&
    (type === "All" || p.type === type)
);

displayProperties(result);

// Save the buyer's search criteria
savedCriteria = {
    location: document.getElementById("location").value,
    budget: document.getElementById("budget").value,
    type: document.getElementById("type").value
};
```

OUTCOME :



RESULT:

The Home Finder website was created successfully. Buyers can search properties by location, budget, and type, save search criteria, add properties to favorites, search agents and companies, and open property locations using Google Maps. Visitor names are displayed in the visitor records section.

EXPERIMENT 12

TITLE: InsureEasy Online Insurance Management System

AIM: To develop an insurance management website that supports admin login, agent registration and approval, customer creation, policy management, customer policy viewing, and premium reminder demonstrations.

PROCEDURE:

27. Create the InsureEasy page with separate sections for admin, agent registration, agent login, customer management, policy management, user login, and reminders.
28. Implement a simple admin login and allow the admin to approve registered agents.
29. Allow approved agents to log in and add customer details such as name, phone number, and email.
30. Allow agents to create policy records with customer name, policy type, premium amount, and due date.
31. Allow customers to log in and view their policy details.
32. Create an SMS reminder demonstration that displays the customer's phone number, policy, premium amount, and due date.

SHORT CODE:

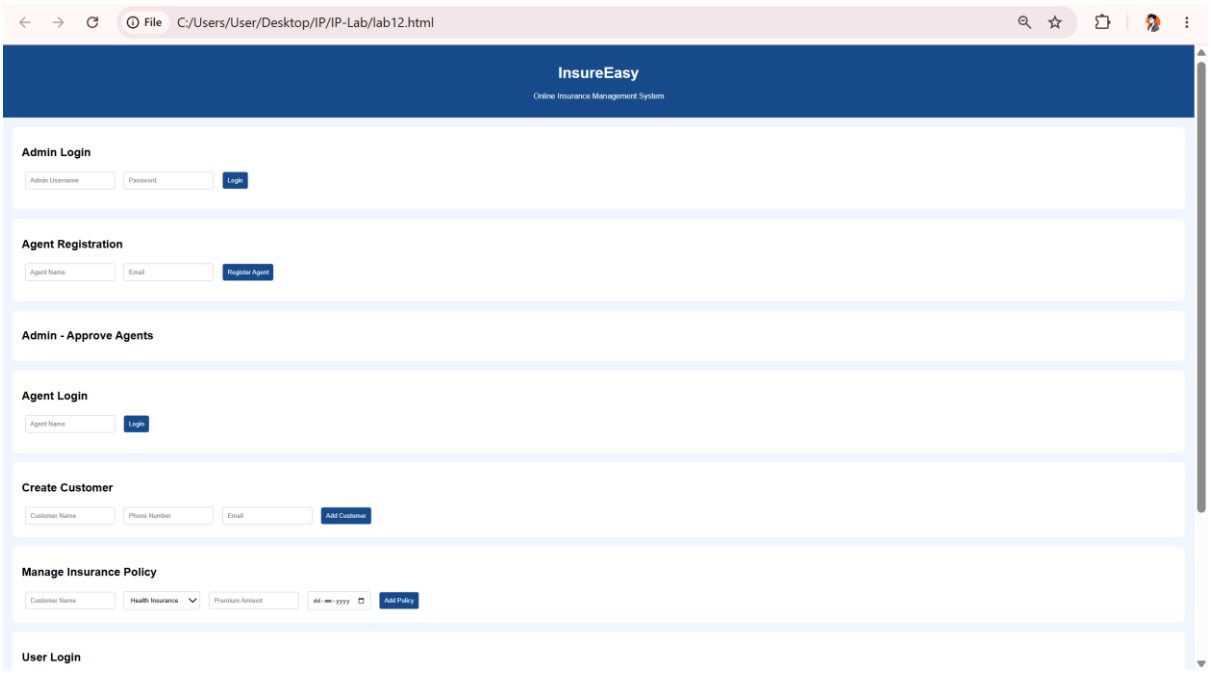
```
// Approve an agent (admin action)
function approveAgent(index) {
    if (!isAdmin) {
        alert("Admin login required!");
        return;
    }

    agents[index].status = "Approved";
    displayAgents();
}

// Add an insurance policy
policies.push({
    customer: customer,
    type: type,
    premium: premium,
    dueDate: dueDate,
    status: "Active"
});

displayPolicies();
```

OUTCOME :



RESULT:

The InsureEasy insurance management website was created successfully. Admin can approve agents, approved agents can add customers and manage policies, and customers can view their policy details. The reminder section demonstrates the premium due alert information.